Vehicle Density Prediction and Counting

Undergraduate Thesis

Submitted in partial fulfillment of the requirements of
BITS F421T Thesis

By

Akash Sonth
ID No. 2015A3TS0265P

Under the supervision of:

Dr. Karunesh K. Gupta

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI, PILANI CAMPUS
December 2018

Declaration of Authorship

I, Akash Sonth, declare that this Undergraduate Thesis titled, 'Vehicle Density Prediction and Counting' and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed: _____

Date: _____

Certificate

This is to certify that the thesis entitled, "*Vehicle Density Prediction and Counting*" and submitted by Akash Sonth ID No. 2015A3TS0265P in partial fulfillment of the requirements of BITS F421T Thesis embodies the work done by him under my supervision.

Supervisor
Dr. Karunesh K. Gupta
Associate Professor,
BITS Pilani, Pilani Campus
Date:

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI, PILANI CAMPUS

Abstract

Bachelor of Engineering (Hons.)

Vehicle Density Prediction and Counting

by Akash Sonth

Traffic surveillance is an important aspect of road safety these days as the number of vehicles is increasing rapidly. The problem of detecting and counting objects from images is something that has been around for quite some time. Deep learning architectures such as convolutional neural networks have been used to predict the density maps. Predicting the occupancy of parking spots has also been done based on individual segmented parking spot images. In this thesis, an algorithm to count the vehicles from low-resolution images is presented. Density maps are first predicted using a convolutional neural network. The neural network is first tested on synthetic data to predict the location of characters inserted onto an image. The network is then used to predict the density maps of vehicles in traffic images. License plate localization is also carried out using the same network architecture. Predictions for the possible vehicle locations is then done by finding the maxima in the density map. The predictions are then compared against the ground truth to obtain the confusion matrix and ROC curve. The time taken for making predictions on an image of size 400×300 is less than a second when the stride is 18 pixels.

Acknowledgement

It gives me great pleasure and pride to express my gratitude for my supervisor Dr. Karunesh K. Gupta for his guidance and support throughout the period of my work. I am grateful to him for giving me the opportunity to work at the Graph Lab, Brno University of Technology.

It gives me immense pleasure to express my gratitude for my guide Prof. Adam Herout for his expert, inspiring guidance and valuable suggestions throughout the period of my time at Brno University. I consider myself fortunate to be associated with him and his research group.

I am sincerely thankful to Dr. Vinod Kumar Chaubey, Head of Department, Electrical & Electronics Engineering, BITS-Pilani, Pilani and Dr. Suresh Gupta, Associate Dean, Academic Undergraduate Studies Division, BITS-Pilani, Pilani campus for their co-operation and encouragement for completing my research work at the Brno University of Technology. I would also like to thank Dr. Navin Singh, Chief Warden, BITS-Pilani, Pilani and other administrative departments for granting me permission on a very short notice.

I would like to express my heartfelt gratitude to my labmates Jakub Špaňhela and Vojtěch Bartl, and other research scholars in the Faculty of Information Technology for the time they had spent for me and making my stay at Brno a memorable one. I take this opportunity to thank one and all for their help directly or indirectly.

I am indebted to my parents Meenakshi Sonth and Prakash Sonth, and my sister Neeti Sonth for their blessings and for their affectionate encouragement and co-operation during every part of my life. Without their constant support it would have been impossible for me to be where I am at present.

My heartfelt thanks and deep sense of appreciation to all the people mentioned here and others whose names I may have missed out.

Contents

Abbreviations

CNN	**C**onvolutional **N**eural **N**etwork
CPU	**C**entral **P**rocessing **U**nit
GPU	**G**raphics **P**rocessing **U**nit
HD	**H**igh **D**efinition
ROC	**R**eceiver **O**perating **C**haracteristic
ROI	**R**egion **O**f **I**nterest
TPR	**T**rue **P**ositive **R**ate

Chapter 1

Introduction

Traffic surveillance is an important aspect of intelligent transportation systems. Vehicle density prediction is very essential to know the times of the day at which the traffic is most at different road sections. The density maps can be further used to estimate the number of vehicles present. Although this may not be of much use on roads, it would be quite useful in places where we would like to know the number of vehicles present at a point of time such as parking lots.

In this thesis, the use of pictures taken from a low-resolution camera to predict the density maps and further use it get individual vehicle locations has been presented. The pictures are taken from buildings and overhead bridges near the parking places. The datasets used in this thesis are taken from [2, 3, 11], and are of high-resolution. They are resized to a lower resolution to increase the processing speed. Patches from these resized images are passed as input to a CNN. The output density maps are then stitched together to match the size of the input image. Location of the vehicles is estimated from the density map by iteratively finding the global maximas and deleting its neighbourhood. Confusion matrix for each image is calculated and a modified ROC curve is plotted for the test data.

The organization of this thesis is as given: Chapter 2 explains the related work which consists of license plate localization and car parking detection. Chapter 3 details the software, programming languages and hardware used during the thesis work. The complete devised algorithm is explained in chapter 4 along with the network architectures used. The results obtained for different datasets are shown in chapter 5. Chapter 6 presents the conclusion.

Chapter 2

Related Work

2.1 License Plate Localization

License plate localization is an important part of traffic surveillance. Existing algorithms take in the entire image and predict 4 values- top-right coordinates, width and breadth [7]. Other methods search for objects of various sizes considering a minimum grid size. It then eliminates those regions having low accuracy [6, 10]. Image processing methods have also been established, but they have a tendency to predict false positives [9, 12].

The neural network architecture constructed for vehicle density prediction is used to predict the location of license plate in image. This would be done by predicting a density map in the region where license plate may be present instead of the entire vehicle. The resulting location can then be used for character segmentation and recognition.

2.2 Car Parking Detection

Finding vacant parking places has always been a time taking and tiring activity. Several methods have been proposed such as [13, 5] which aim to find vacant parking places, or those that would be unoccupied in some time. Existing parking vacancy detection algorithms such as [3] detect whether a parking space is vacant or not based on their own dataset and the public dataset [1]. These require the parking regions to be segmented beforehand. Other methods involve placing a sensor at each and every parking spot which turns out to be very expensive.

The network architecture which is borrowed from [8] predicts the traffic density and not their individual locations. The original annotation are converted to density maps using a Gaussian filter with standard deviation of 15. Predicting the count from such a density map is almost impossible due to no distinction between adjacent vehicles.

In this thesis, we present an algorithm which uses a density map generated with a low standard deviation to make predictions of the vehicle locations. This is mainly done by finding the maximas less than a threshold, and deleting their neighbourhood from the density map.

Chapter 3

Tools and Equipment

3.1 Softwares and Languages Used

3.1.1 MATLAB

MATLAB is a programming language and tool developed by MathWorks. The generation of density maps from annotations was done using the MATLAB tool. It was also used to generate the masked images from the original given images and the .mat mask file in the TRANCOS dataset.

3.1.2 Python

Python is an open source language. It was used for viewing the data, network training, making the predictions, and calculating the metrics. The vast array of libraries in Python and the fact that they are all open source is what makes Python so developer friendly-

NumPy

The NumPy library was mainly used for numerical computations and all the array based operations.

SciPy

SciPy was used for applying filters on images while finding the maxima.

OpenCV

All the image reading, writing and modifications were done using OpenCV. The multitude of computer vision functions present in this library makes it one of the most popular libraries for images.

matplotlib

Matplotlib was used for all curve and graph plotting tasks. It was also used for viewing images in the Jupyter notebook. Viewing multiple images one over the other with transparency was all done quite easily with this library.

Keras

The neural network models were all built and trained using Keras. It also allows us to load trained weights onto a model and retrain a model. In the end, we can save the weights for future use and testing. The Keras libary is capable of running over TensorFlow.

3.1.3 Jupyter Notebook

Jupyter notebook is an open-source browser based environment for programming. Each file/page of this notebook is run on a kernel. The page of the notebook is divided into cells where each cell can be executed independently. The advantage of Jupyter notebooks is that results and plots can be viewed simultaneously along with the code. These notebooks can also be shared others.

3.1.4 VUT Annot

VUT Annot is an Android application developed by the Faculty of Information Technology, Brno University. It is used for the purpose of annotating vehicles or objects in an image. The annotations can be done in the form of single dots or strikes. The app also allows zooming into the images while annotating. Some other features of the app are undo operation and deleting a selected annotation.

3.2 Hardware Used

Graphic Card

The GPU used for training and testing the networks is a Nvidia Geforce GTX 980 having 4GB of memory.

Camera

The images taken personally at the car parking lot are from a Motorola Moto G4 Plus having a 16-megapixel primary camera .

Chapter 4

Algorithm

4.1 The Counting CNN Model

The prediction of density maps from an image is done using the CCNN model as defined in [8]. The architecture of the same is shown in 4.1.

The architecture consists of 6 convolutional layers which are named as Conv followed by the layer number. Conv1 and Conv2 layers have filters of size 7x7 with a depth of 32, and they are followed by a max-pooling layer, with a 2x2 kernel size. The Conv3 layer has 5x5 filters with a depth of 64, and it is also followed by a max-pooling layer with another 2x2 kernel. Conv4 and Conv5 layers are made of 1x1 filters with a depth of 1000 and 400, respectively. Finally, Conv6 is another 1x1 filter with a depth of 1. Each of the convolutional layers in the network uses ReLU (Rectified Linear Unit) as the activation function. The ReLU activation function curve and an example of max-pooling are shown in Fig. 4.2 and Fig. 4.3 respectively.

FIGURE 4.1: The Counting CNN architecture

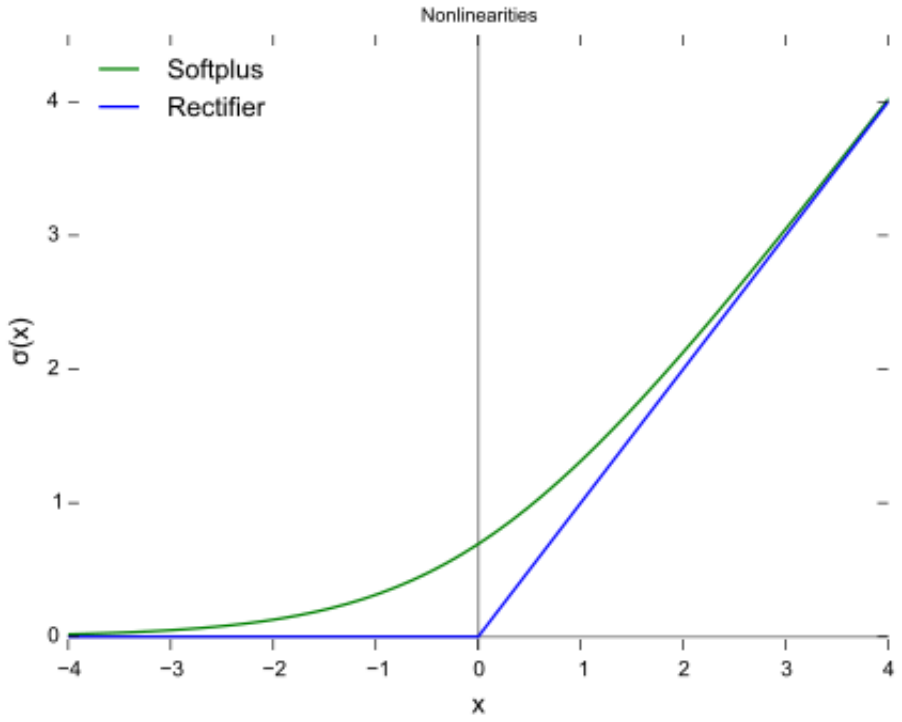


FIGURE 4.2: ReLU activation function

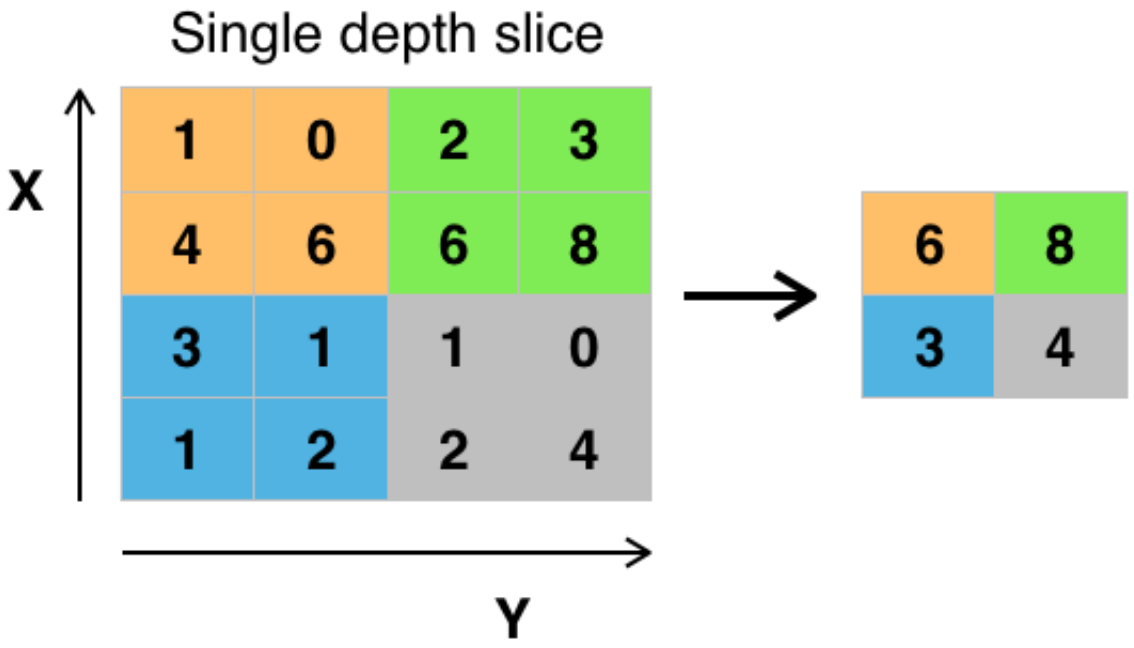


FIGURE 4.3: Max-pooling [15]

4.2 The Hydra CNN Model

The Hydra CNN model defined in [8] is a multi-scale model as opposed to the scale-aware CCNN network. This is because of the multiple inputs present in the network which crop out images of different scales from the original input patch.

Each of the input heads of this network have a similar architecture as the CCNN model. The only difference is that the final Conv6 layer is replaced by a flattening layer. All the flattened layers are concatenated and is then followed by 2 dense layers of 512 nodes each with the ReLU activation function. Finally, a dense layer with 324 nodes serves as the output having a linear activation function.

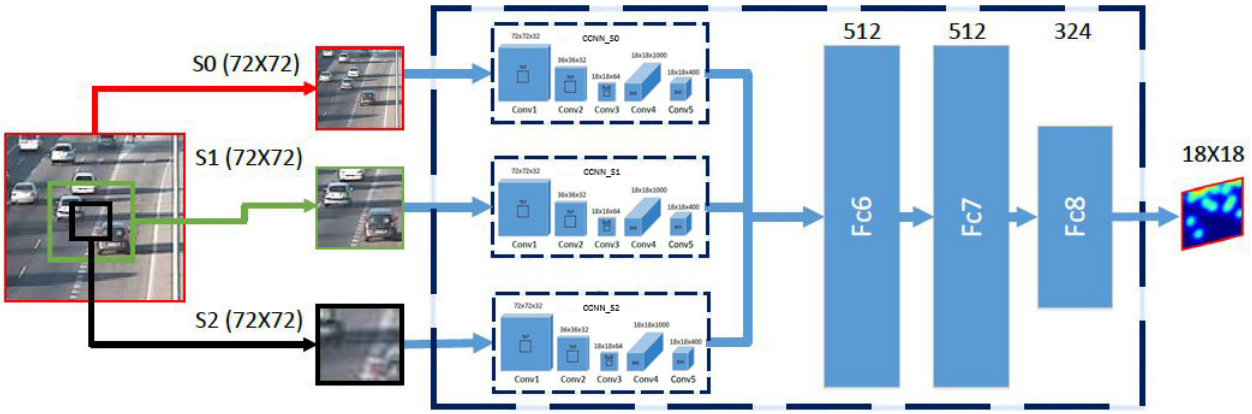


FIGURE 4.4: The Hydra CNN architecture

Although this network is supposed to be more accurate than the CCNN model, its complexity and number of trainable parameters results in long training and testing times. This in turn reduces the efficiency of the algorithm as similar results can be achieved by training the CCNN model with different scaled images.

4.3 Image Stitching

When the original image size is more than the network input size, it is zero-padded, and patches from this are passed as input. The output density maps are resized to match the size of the network input. These and are then stitched together and cropped to give a complete density map having size as the original image. The number of patches obtained from each input image depends on the stride and the original image size. Stitching is done by averaging all the overlapping intensity values at any pixel location. Several other methods of stitching the overlapping regions such as median, maximum value, and minimum value are used. The best results are obtained by averaging over all the other methods for this case. The number of images overlapping at a position increases from the edges to the centre of the image or some position before that.

4.4 Annotation Estimation

The predicted density map has to be converted to an annotation map so that the number of objects can be counted. This is done by locating the maximas in the predicted density map. The methods used to find the maximas corresponding to the objects are given below-

Finding the Local Maximas

All the local maximas on the 2D surface plot of density maps are found which are above a threshold value. These maximas would correspond to the objects. This method leads to multiple misses even when the object size is considerably large in the image.

Finding the Global Maxima and Deleting its Neighbourhood

The global maxima in the density map is found and compared against the threshold. If it is greater, a rectangular region surrounding and including the point is deleted from the density map. This is repeated till there is no maxima larger than the threshold. Although the results obatined by this method are better than those obtained by the local maxima, deleting rectangular areas ends up deleting the maximas of adjacent objects when the objects are very small in size.

To tackle this problem of information being lost, all the pixel values covered by a small elliptical region surrounding the maxima are summed up. Ellipses with orientation ranging from 0 to 180 degrees in intervals of 1 degree are chosen. The orientation at which the sum of pixels is maximum, corresponds to the orientation of the object. An elliptical region at this orientation is then deleted from the density map at the maxima location. The interval is further increased to 5 degrees to speed up the counting algorithm.

4.5 Calculating the Confusion Matrix

A hypothetical circle of a defined radius is drawn around each of the estimated object locations. If any ground truth annotations exist inside any of the circles, then that estimated point is a true positive. Else, it is a false positive. All the annotations which are missed out i.e., they did not fall inside any region would be the false negatives. Calculating the true negatives is meaningless in this scenario as it would mean counting each and every pixel in the image which do not have annotations corresponding to either the ground truth or prediction.

Similar objects differ in size based on their distance from the camera. Circular regions of varying radius are therefore drawn with radius increasing linearly from the top of the image to the

bottom. Although this is more accurate, another undying problem is to be solved. Trying to count the true and false positives in this way is faulty. This is because, those annotations which are present in more than one circular region get counted twice as true positives. Once any annotation is been found to be a true positive, the corresponding ground truth annotation should be deleted so that it does not get counted again. This is done for each and every estimated annotation so that we get the accurate count.

Metrics such as precision, recall and true positive rate are then calculated. These metrics are used to plot the ROC and precision-recall curves which give us an idea of how good the prediction is. ROC curve is modified in our case. Athough the TPR is taken on the y-axis, only false positives are considered on the x-axis. This is because we don't have an estimate of the true negatives.

Precision (P) and recall (R) are given by the following equations: Here T_p represents the true positives, F_p represents the false positives, and F_n represents the false negatives. Recall is also known as the true positive rate (TPR) .

$$P = \frac{T_p}{T_p + F_p} \tag{4.1}$$

$$R = TPR = \frac{T_p}{T_p + F_n} \tag{4.2}$$

Using this method, we get scatter plots instead of curves for ROC and precision-recall where each point corresponds to an image. To get the curves, the true positives of all the images are summed up. Similar operation is done with the false negatives and false positives. The new value of TPR is now found. The threshold value is changed and this operation is repeated for different values. The metrics at these thresholds are then plotted to get the required curves.

Chapter 5

Results

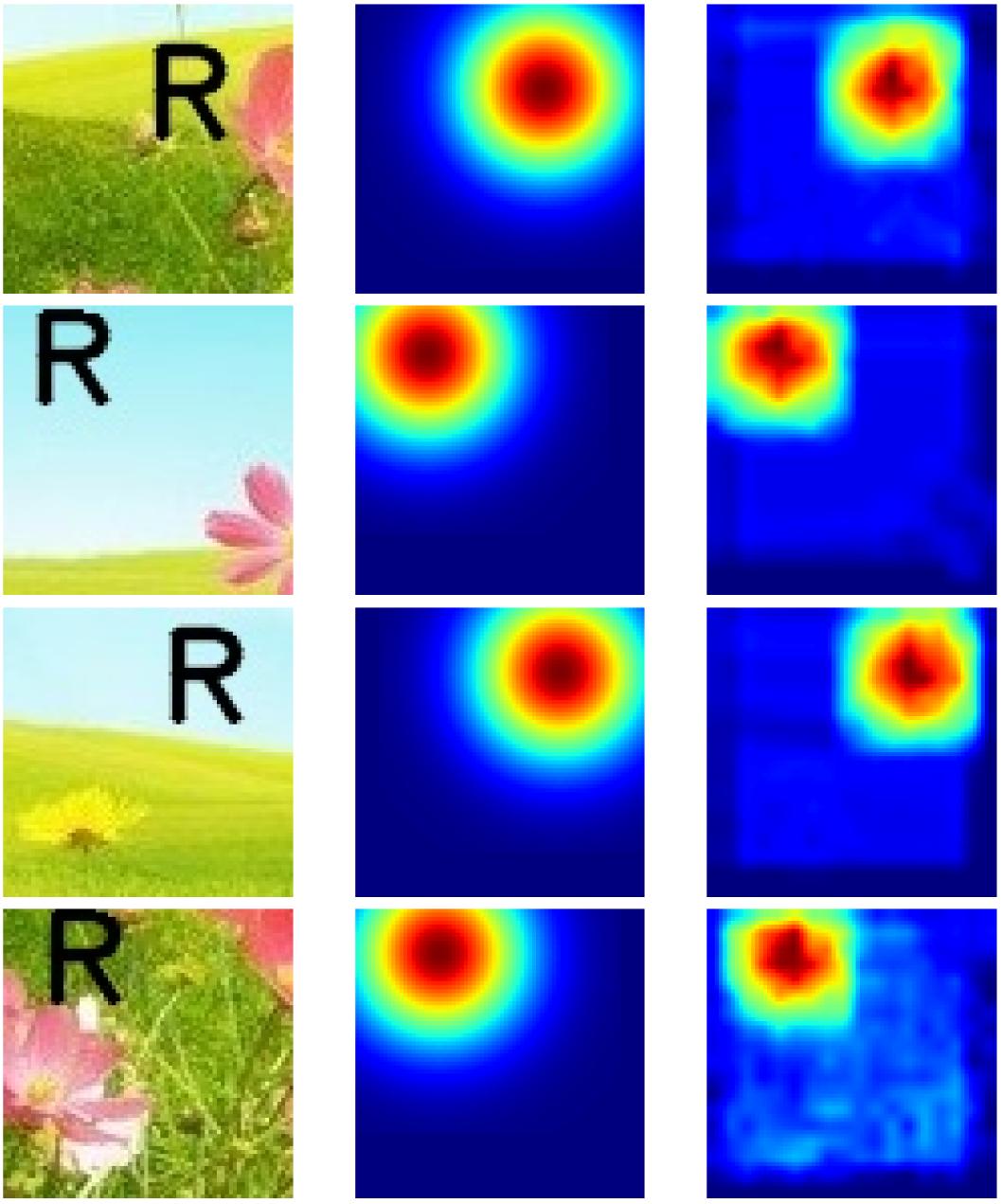
5.1 Synthetic Dataset

Basic Background

An image with almost uniform and light colours throughout the image is chosen as the background (see Fig. 5.1). Patches of size 72×72 are extracted from this background image, and the character 'R' is synthetically inserted onto them at random locations. The annotations for the ground truth representing the center of the inserted character is also generated. The ground truth is then prepared from the annotations by a Gaussian filter with a standard deviation of 15.

FIGURE 5.1: The basic background chosen for synthetic data generation [14]

In total, 200 patches are generated with 80 as training, 20 as validation, and 100 as testing. The CCNN network is trained and tested on this dataset. This is done to check whether the model learns on a small and simple data and would therefore be able to perform satisfactorily on larger datasets. The training is done on a CPU with 8GB of memory for 100 epochs.



(a) (b) (c)

FIGURE 5.2: (a) Input image, (b) Ground truth, (c) Prediction after training 100 images for 100 epochs

This low number of epochs and small training data is sufficient because of the easily distinguishable foreground and background. This is confirmed by the density map predictions obtained on the test data as shown in Fig. 5.2.

Complex Background

A tougher dataset would give a better idea of how robust the network is. Therefore, a complex image with multiple colours and patterns is now chosen as the background (see Fig. 5.3). The synthetic data and ground truth is generated in a similar fashion as earlier.

FIGURE 5.3: The complex background chosen for synthetic data generation [4]

600 patches are generated with the split as- 400 for training, 100 for validation, and 100 for testing. The network is trained on a CPU with 8GB of memory for 100 epochs. This trained model fails to distinguish between foreground and background when the colours are almost similar. A much larger dataset composed of 10000 patches is generated with the split as- 6400 for training, 1600 for validation, and 2000 for testing. This is then trained on a GPU with 4GB of memory for 100 and 500 epochs.

Increasing the data size and the number of epochs has evident improvement in the predictions as shown in Fig. 5.4.

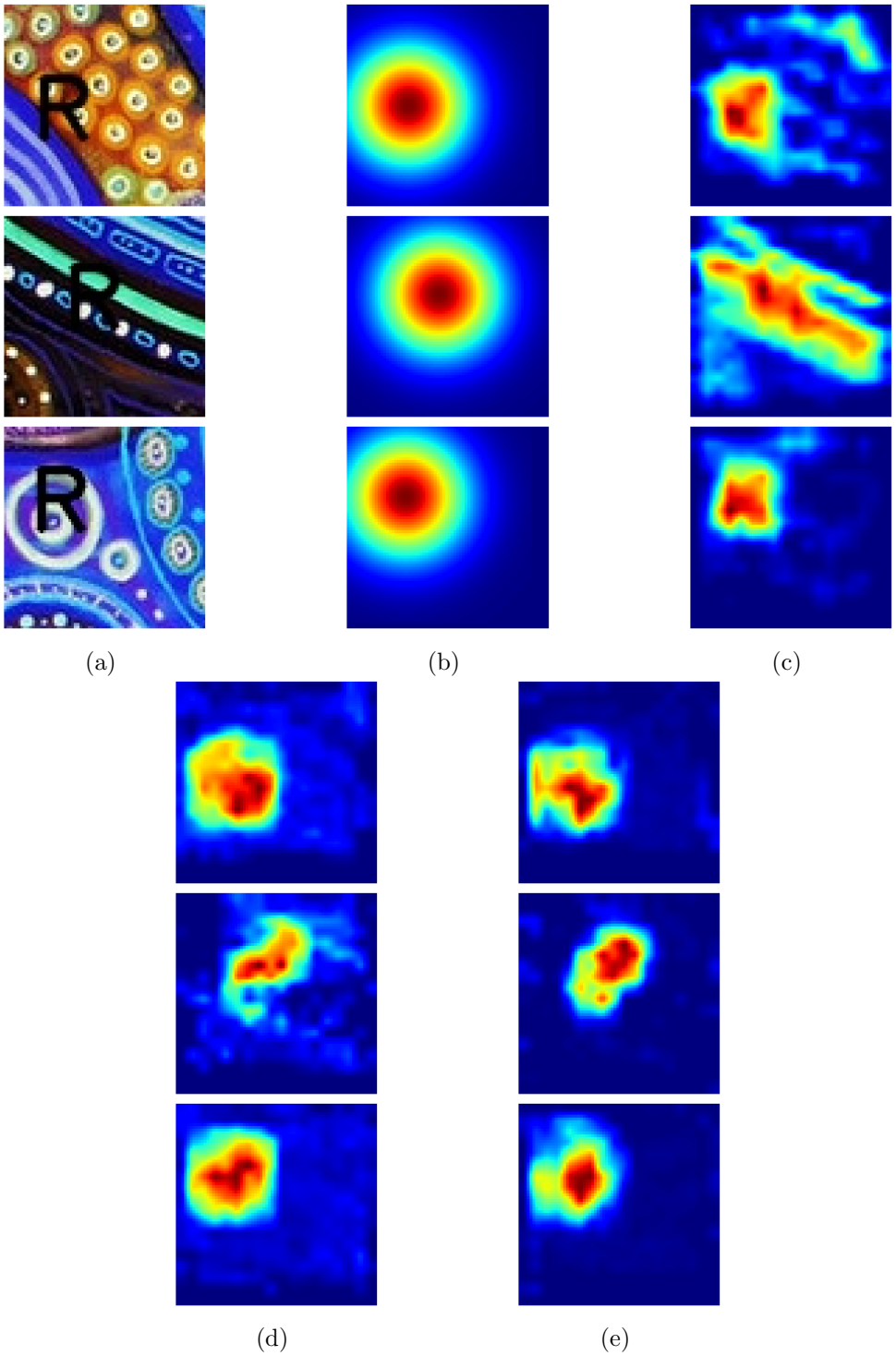
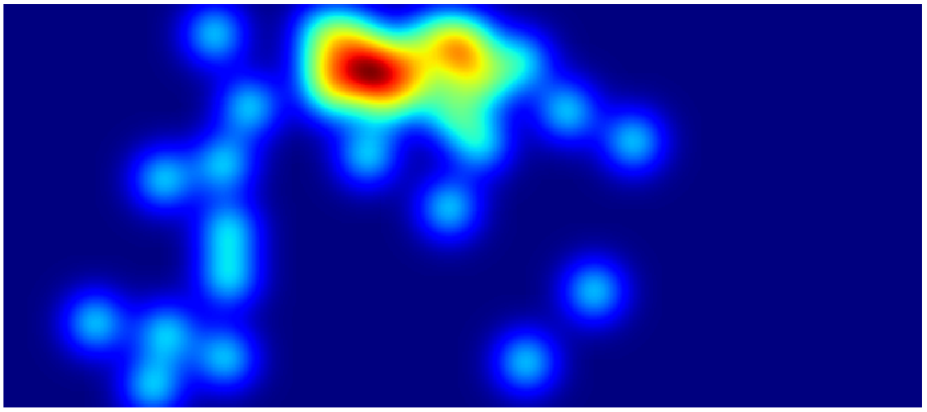


FIGURE 5.4: (a) Input image, (b) Ground truth, (c) Prediction after training 500 images for 100 epochs, (d) Prediction after training 8000 images for 100 epochs, (e) Prediction after training 8000 images for 500 epochs

5.2 TRANCOS Dataset

To test the working of model on real-life data, the public dataset TRANCOS [11] is used. This dataset consists of images of different road sections with different traffic densities. The dataset also includes a mask for each image which crops out a part of the input image and the ground truth. Locations of the vehicles have been marked with annotations in the form of dots. The annotated images are passed through a Gaussian filter with standard deviation of 15 to give the vehicle density maps (Fig. 5.5).

(a)



(b)

FIGURE 5.5: (a) Original image, (b) Ground truth density map

From this dataset, 800 random images are taken, and 20 random patches of size 72×72 are extracted from each image. Of these 16000 total input patches, 12800 patches are used for training, and rest 3200 for validation. The CCNN network is trained for 1000 epochs on this data on a GPU with 4GB of memory.

Patches from each test data image are passed on to the network as input with a stride of 6. The output density maps are upscaled to match the network input size and stitched together to give

the final density map. Some of the results on the test data are shown in Fig. 5.6.

Training and testing the network on the TRANCOS dataset is done purely to replicate the work given in [11]. Density maps although give a rough idea of the traffic density, they aren't very good for counting the vehicles. This is because the density map is merged for multiple vehicles close to one another. Distinguishing the vehicles is not possible in this kind of data. Also, counting the vehicles would be much useful in places such as parking lots than on roads. Traffic surveillance and calculating the speed of the vehicles would be much more practical than counting the vehicles on the roads.

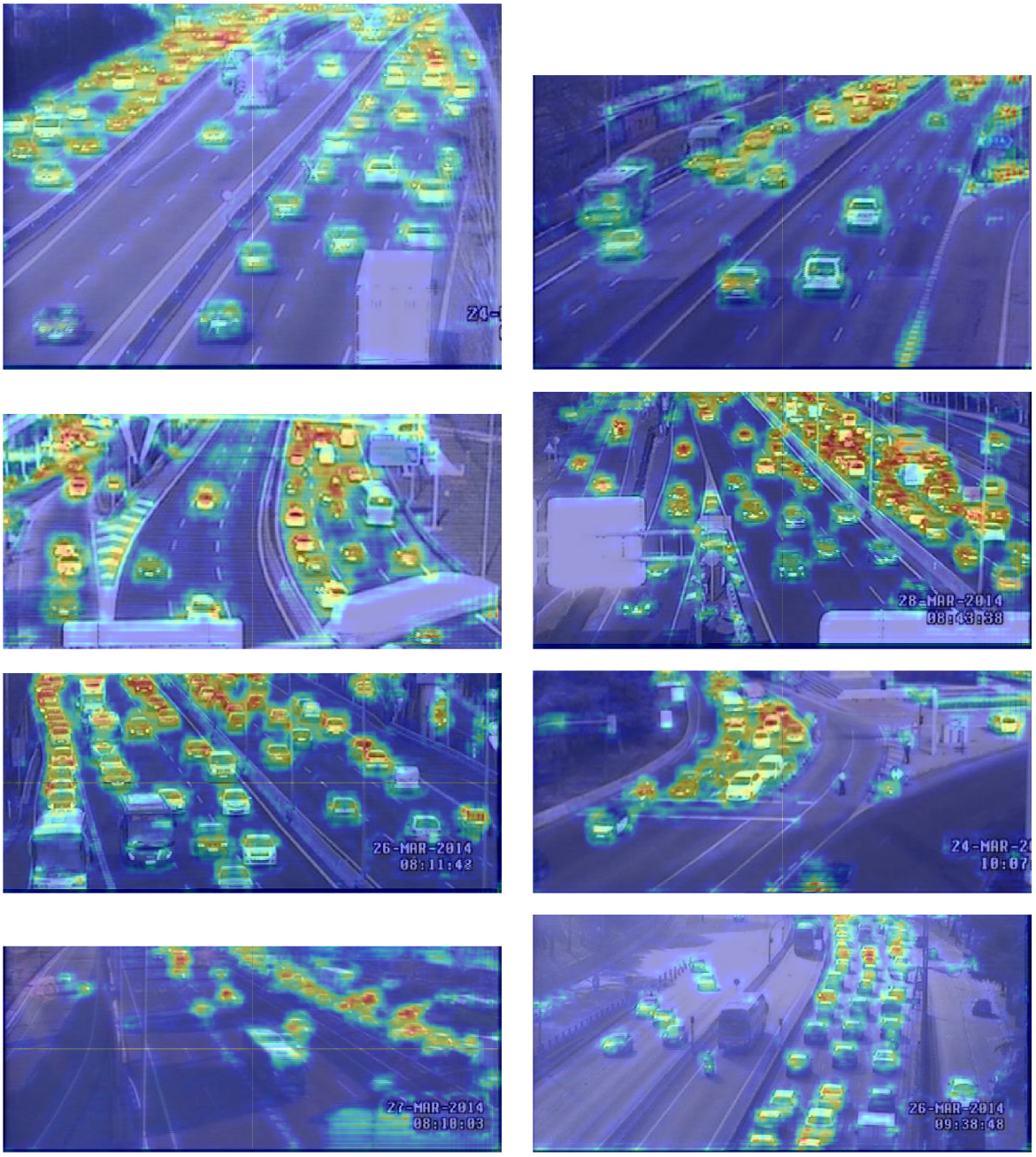


FIGURE 5.6: Predictions on the TRANOCS dataset

5.3 ZoomLP Dataset

ZoomLP is a private license plate localization dataset prepared by the Graph Lab of Brno University. This dataset has been captured by a full HD camera positioned on a foot over bridge over a highway in Brno. The vehicles is this dataset are much closer to the camera and are quite large in size as compared to TRANCOS. The images are captured at different times of the day with different positions of the camera.

The network is trained in a similar manner as TRANCOS. The only difference is that the density maps correspond to the license plates instead of the vehicles. The aim of the network is to predict the location of the license plates. The original images are also resized from 1920×1080 to 480×270. The dataset consists of 21 folders having 4000 images each. Each folder has images captured from different position of the camera, and at different time of the day. The model is only trained on images from one of these folders. The test data consists of images from each of these folders. The predictions on the test data are shown in Fig. 5.7.

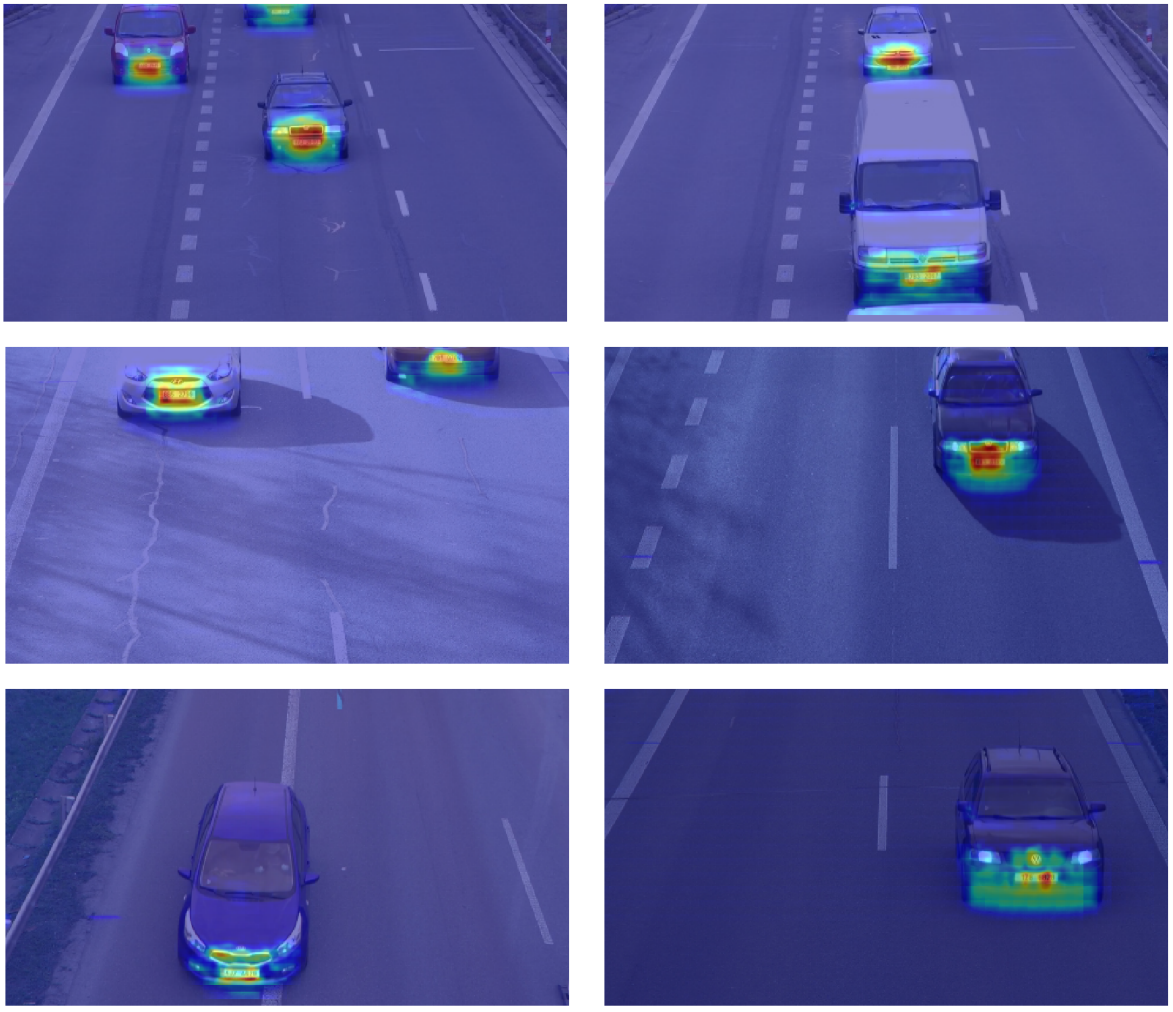


FIGURE 5.7: License plate localization and the original input image

5.4 CarPark Dataset

CarPark is a private license plate localization dataset prepared by the Graph Lab of Brno University. This dataset has been captured by a full HD camera from public accessible buildings near the parking lots. The dataset includes annotations in the form of strikes for big vehicles, and dots for small or occluded vehicles. The annotations were prepared using the VUT Annot application.

The organization of the results in this section is as follows- ROC curves are plotted for different variations of the algorithm. The ROC curves represent the timeline of how the algorithm improves with each variation. 200 test images have been considered while plotting the ROC curve. Each of these curves has been plotted for the threshold intensity values of 30, 45, 60, 75, 90, 105 and 120. Those graphs having multiple curves are for different strides whose values are mentioned in the legend of the corresponding graph. The density map is predicted based on the CCNN model trained with different parameters. The annotations are estimated for each predicted density map, and the corresponding ROC curve is plotted based on the algorithm devised in Chapter 4. The red dots in the predicted annotations correspond to the true positives, the black dots correspond to the false positives, and the green strikes and dots are the ground truth annotations.

Local maximas that should correspond to the vehicle locations are found by considering a filter size of 30 pixels. To verify this prediction, ground-truth annotations are located in a search radius of 10 pixels from the maxima. The resultant ROC curve is shown in Fig. 5.8.

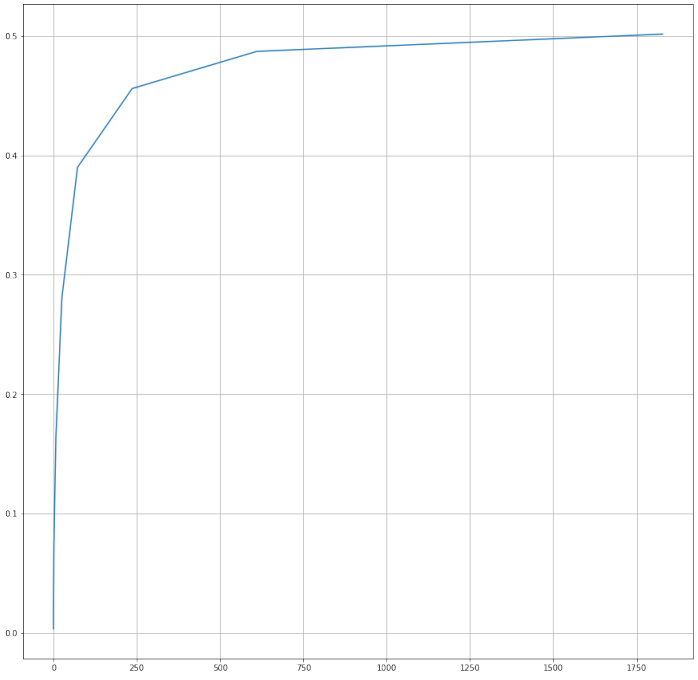


FIGURE 5.8: ROC curve for a fixed search radius of 10pixels

A fixed radius as considered above implies that the vehicles are of the same size throughout the image. This is not true as the size of the vehicle viewed varies with the distance from the camera. The search radius is therefore varied linearly from 5 pixels at the top of the image to 50 pixels at the bottom of the image (Fig. 5.9).

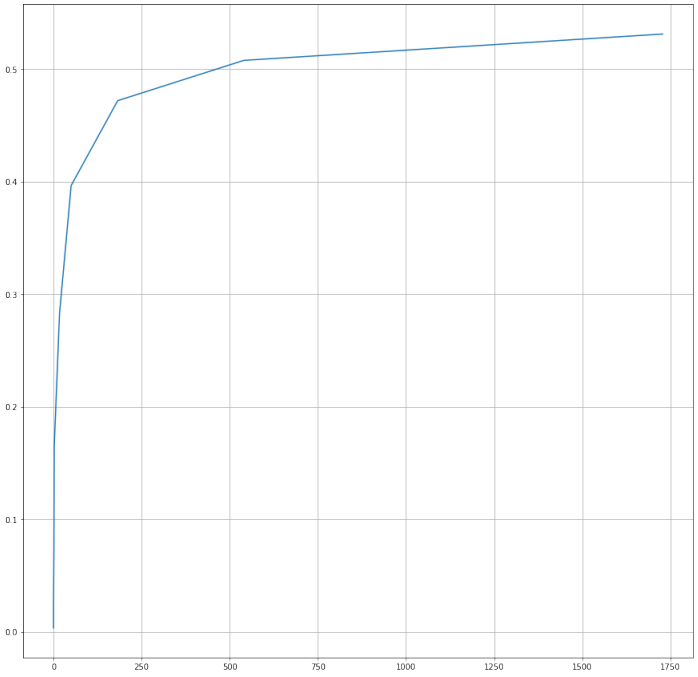


FIGURE 5.9: ROC curve for a linearly varying search radius

Top 10% of the image is masked as the vehicles in that region are too tiny for even the human eye. This region would region leads to a high number of false positives. The corresponding ROC curve is shown in Fig. 5.10.

Using local maximas to find the vehicle locations leads to several misses i.e., false negatives. The method of finding the vehicle location is therefore changed to iteratively finding the global maxima in the density map and deleting a rectangular region around it (Fig. 5.11).

The predictions are verified by searching for the ground truth annotations in a search radius from a predicted vehicle location. Several times, there are more than one predictions which point to the same annotation. This leads to a higher true positive value than actually present. This is also the reason for the ROC curve crossing the value of 1 on the TPR axis as in Fig. 5.11. Therefore, the ground truth annotation is deleted when a corresponding true positive is found. ROC curve after this modification in the algorithm is shown in Fig. 5.12

For density prediction, the network is trained in a similar fashion as TRANCOS. The predicted density map and the estimated annotations can be seen in Fig. 5.13. Using a standard deviation of 15 leads to a high number of false positives.

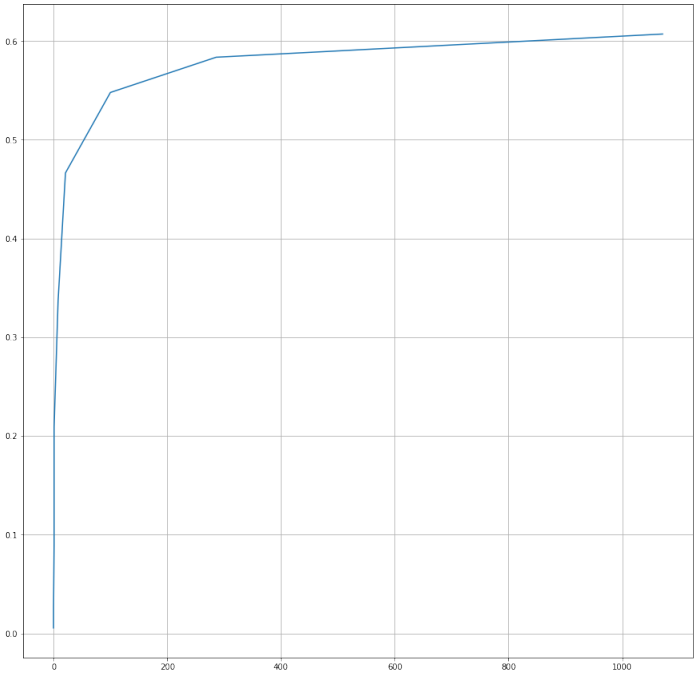


FIGURE 5.10: ROC curve after masking the top 10% of each image

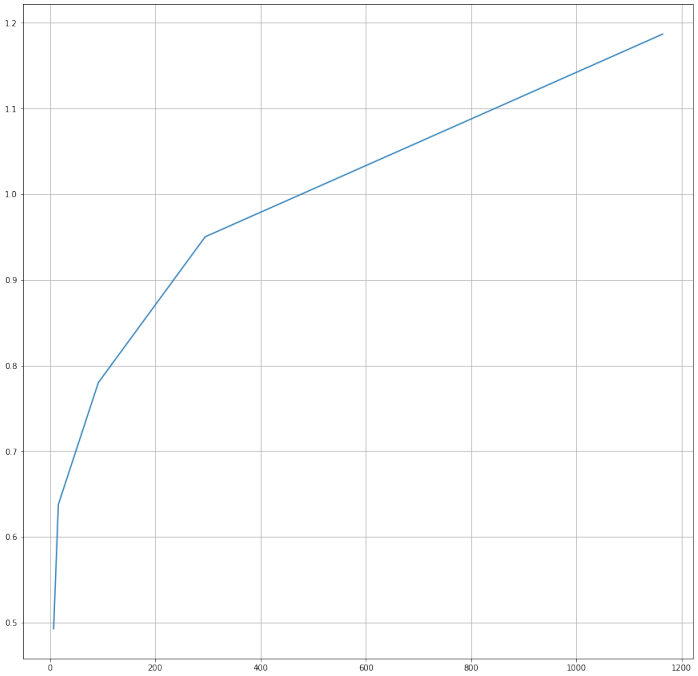


FIGURE 5.11: ROC curve for vehicle location found from the global maxima

The aim of the algorithm is to count the total number of vehicles in the parking place as accurately as possible. The top 10% of each image is therefore masked out as the vehicles are very small in that region and cannot be counted even by the human eye.

As seen, standard deviation of 15 is not ideal when the purpose is counting the vehicles. Therefore, a new network is trained with the density maps created by a Gaussian filter with

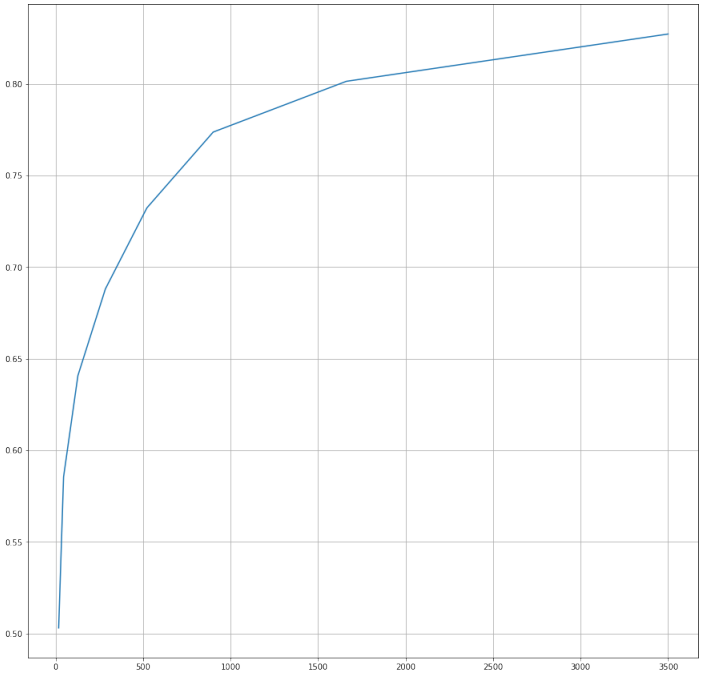


FIGURE 5.12: ROC curve for unique true positives

standard deviation of 2 instead of 15. The predicted density map and annotation is much more distinguishable for individual vehicles than before as can be seen in Fig. 5.14.

One of the methods in trying to optimize the performance was to change the network input size. The input size of the network is increased to 144×144 so that the larger vehicles could be predicted with ease (Fig. 5.15). Although a larger patch size leads to less number of total patches, the training and testing increases due to the input size being 4 times of the original. Moreover, the results do not show any considerable improvement as can be seen in the ROC curves (5.16). Therefore, the network with input size of 144×144 is discarded.

The density map predictions of the smaller vehicles tend to merge even when they are distinct in the ground truth. This leads to a lower count of true positives than actually present. Therefore, the model trained with standard deviation of 2 is retrained by taking patches near the top of the image with a higher probability than those at the bottom. The results are shown in Fig. 5.17.

Data augmentation is one of the methods employed to obtain better results in image based predictions when data is limited. Horizontal flipping is one of the augmentation methods used. Cropping to half the original size and upscaling it back is another augmentation method used. Flipping is also done on this cropped and resized data. This results in a training data that is 4 times bigger than before. The model is retrained again, but this time on the augmented and larger training data (Fig. 5.18, Fig. 5.19).

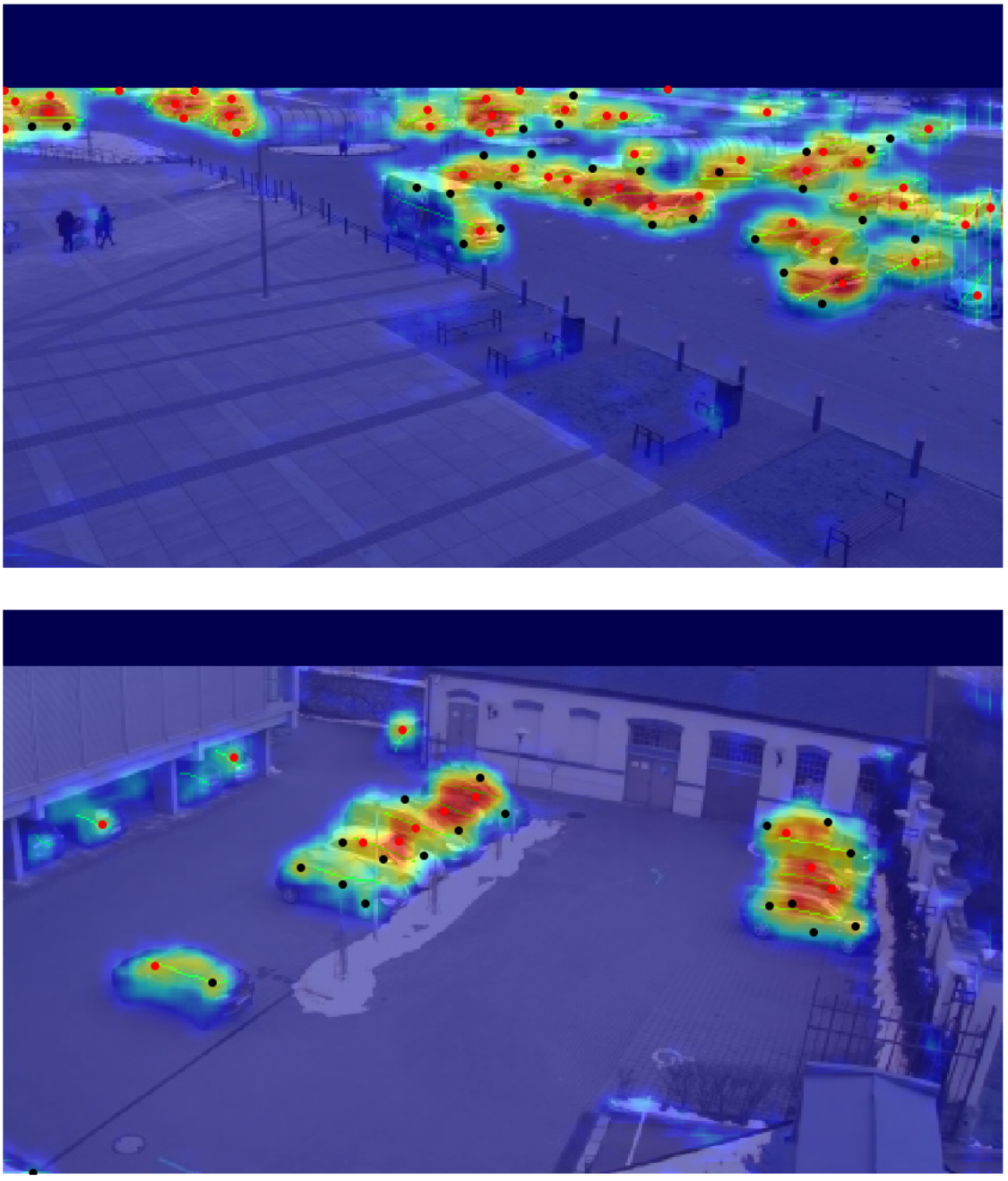


FIGURE 5.13: Predictions for ground truth density map having a standard deviation of 15

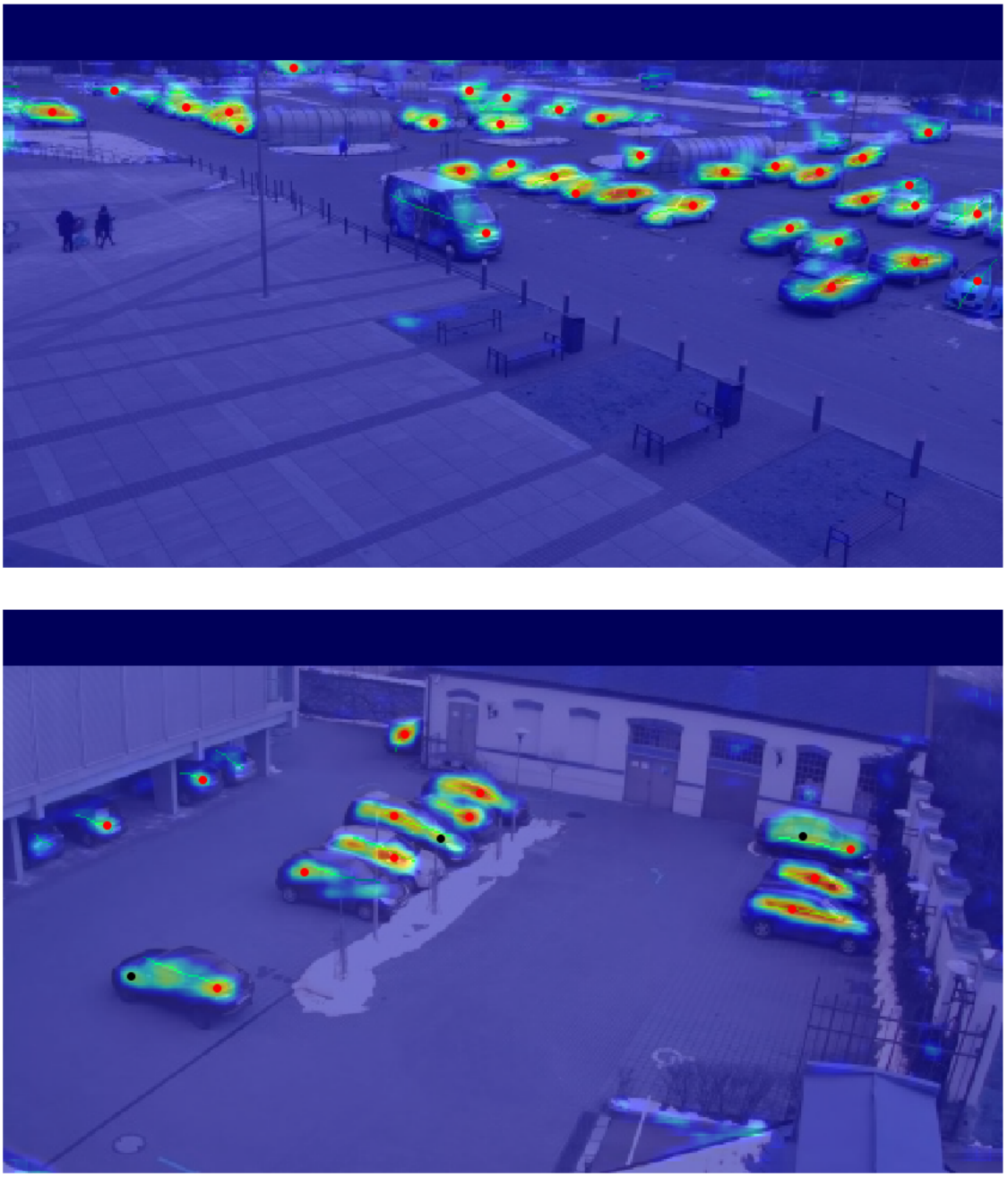


FIGURE 5.14: Prediction for the ground truth density map with a standard deviation of 2

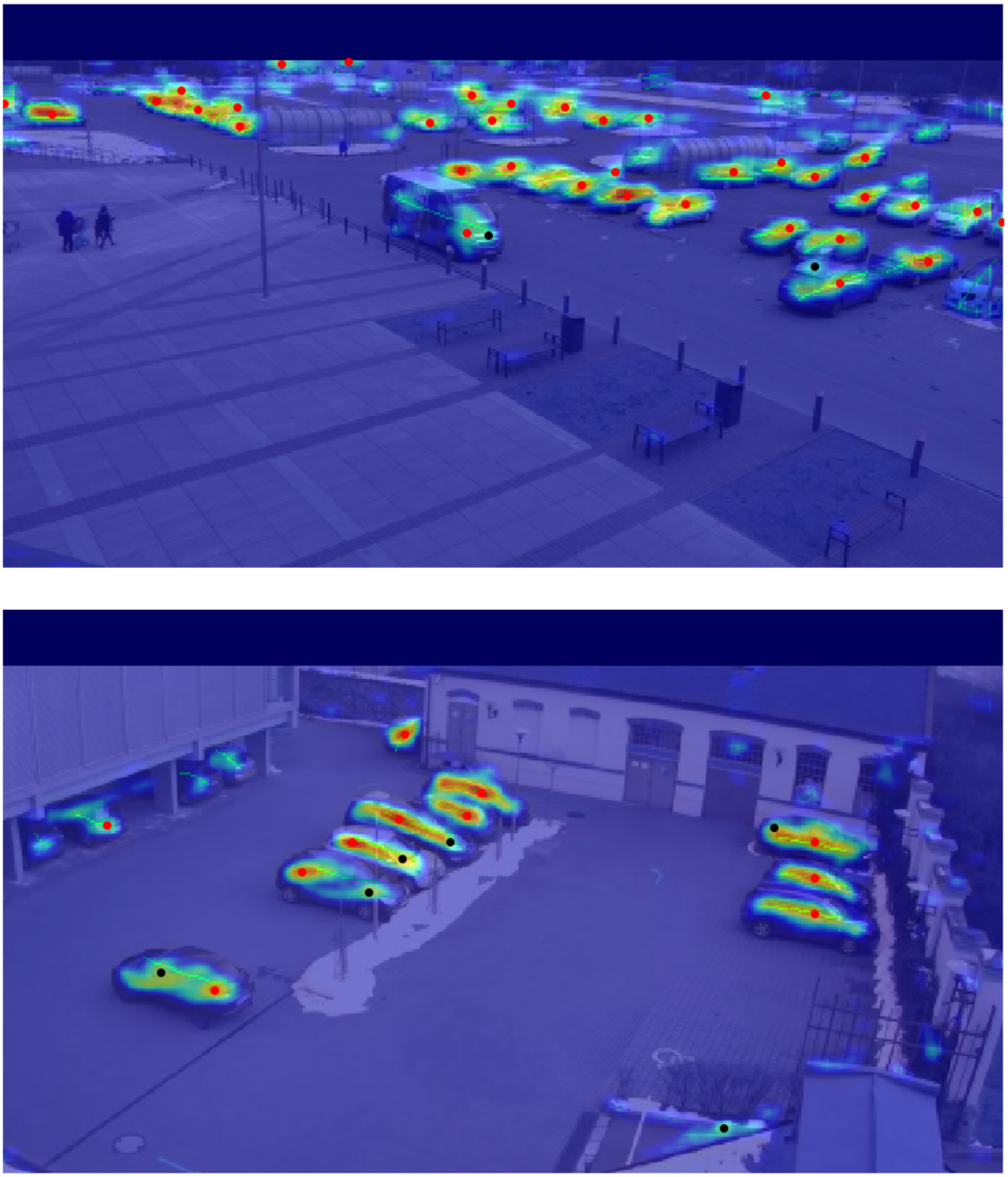


FIGURE 5.15: Prediction for the input size of the network changed to 144×144

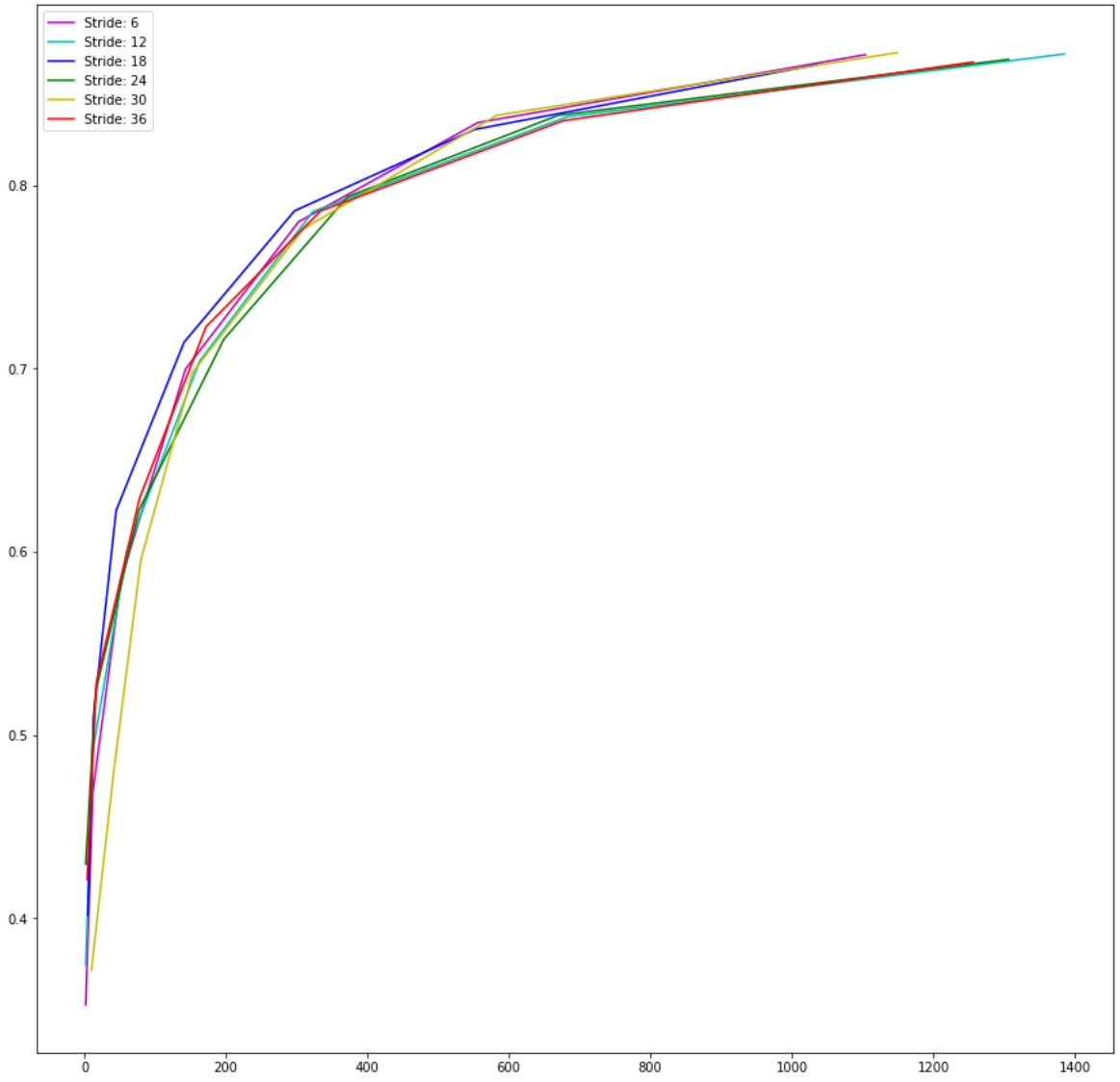


FIGURE 5.16: ROC curve for input size of 144×144

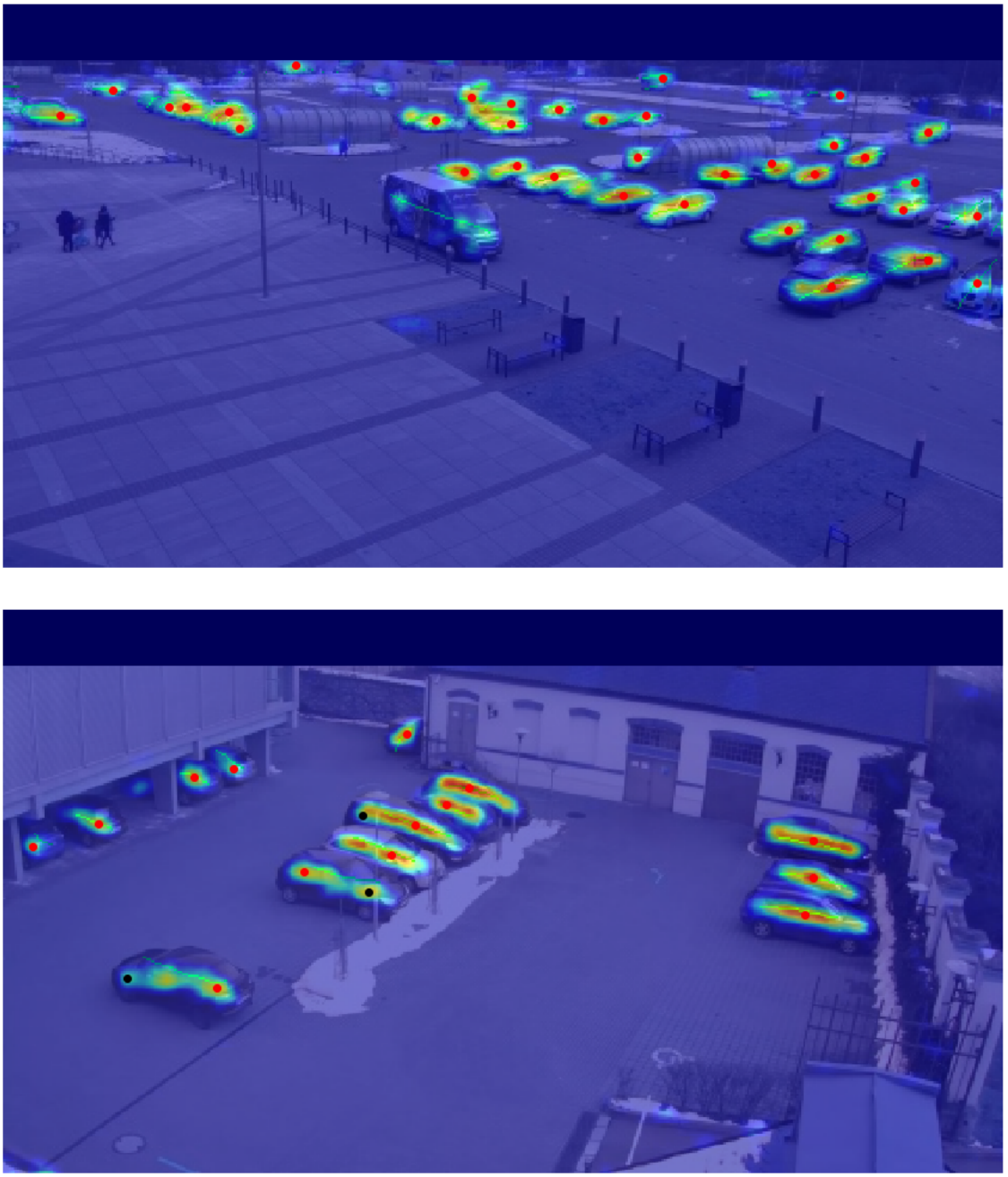


FIGURE 5.17: Predictions after retraining the model on smaller vehicles

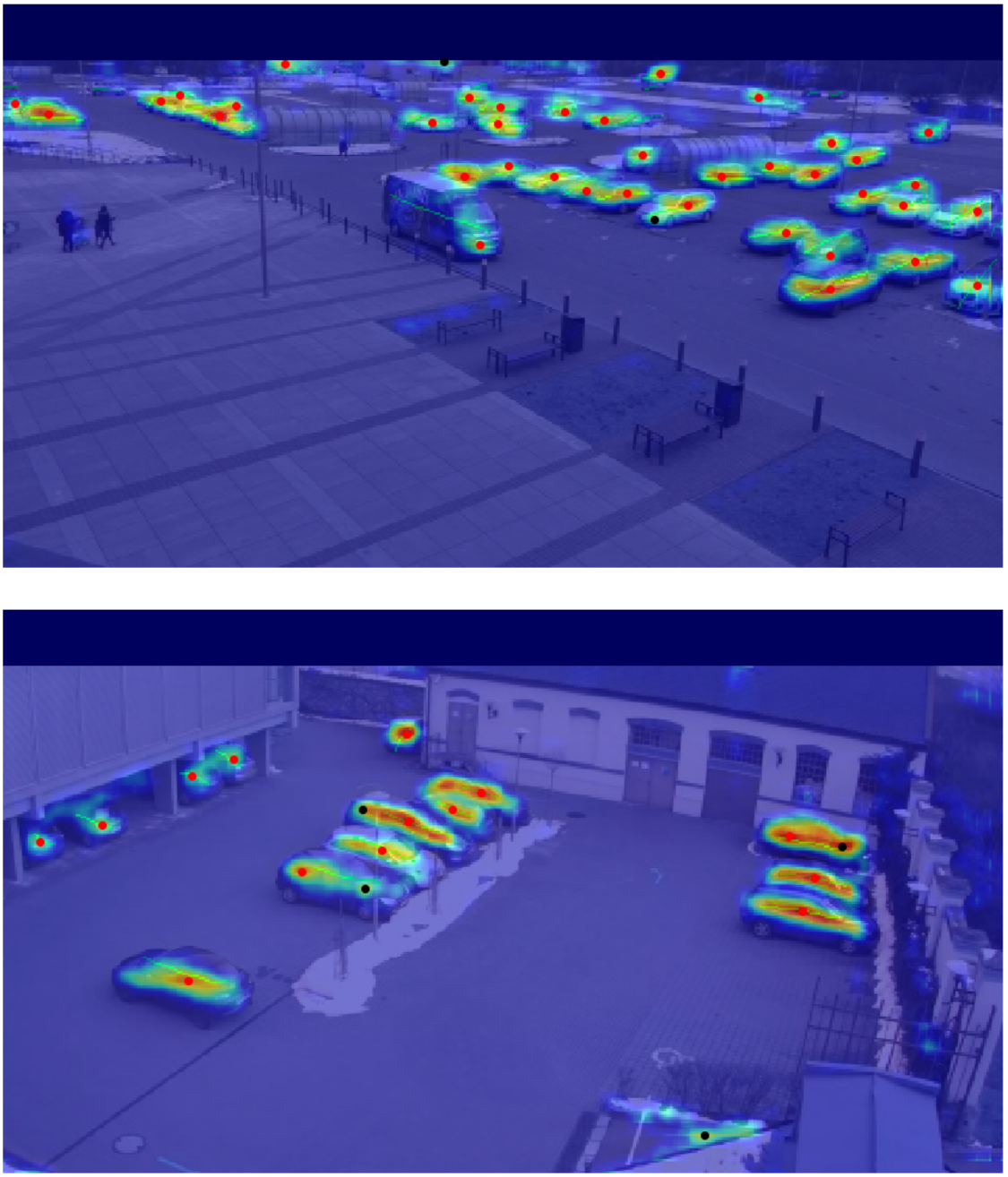


FIGURE 5.18: Predictions after data augmentation

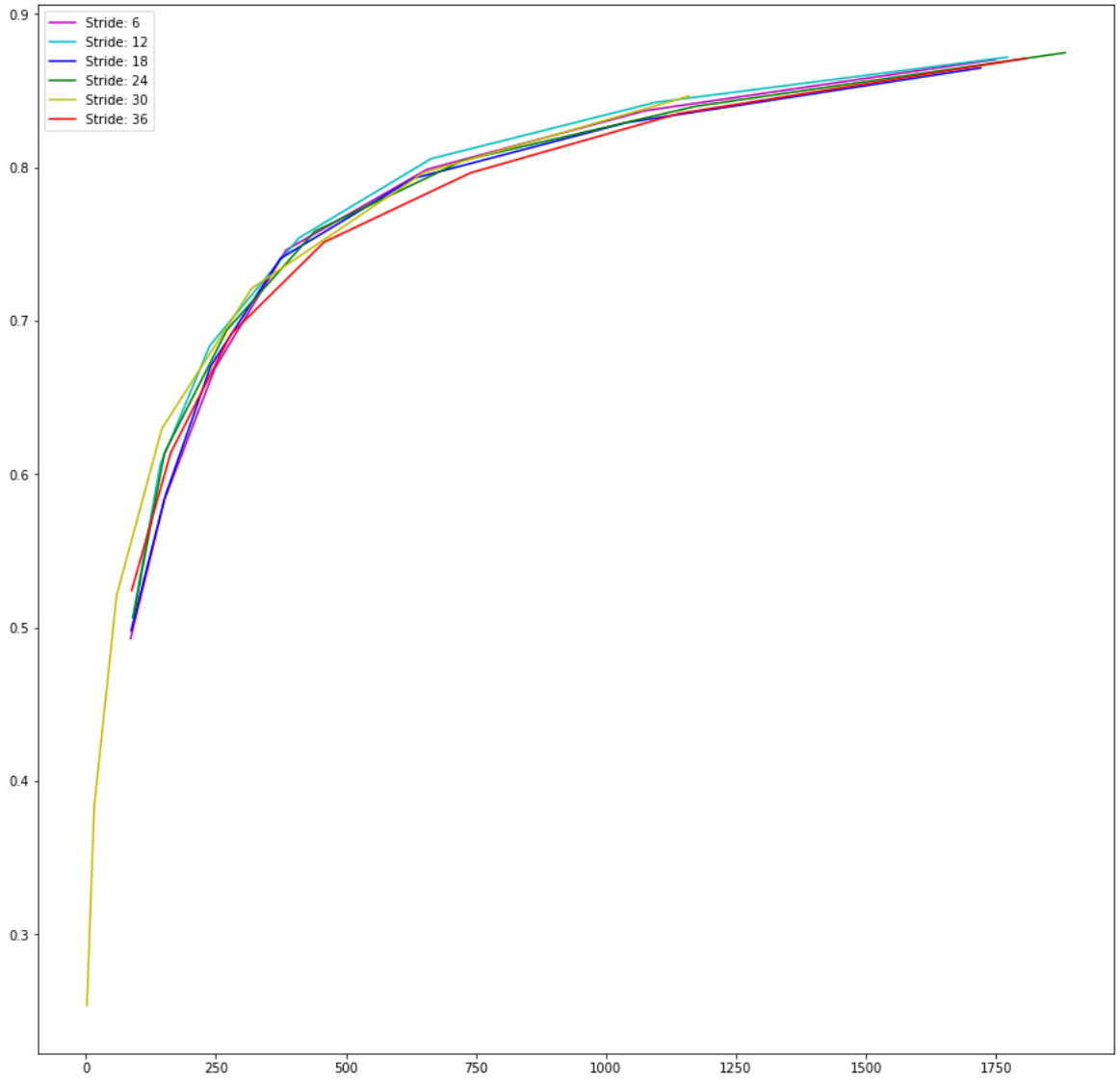


FIGURE 5.19: ROC curves after data augmentation

Chapter 6

Conclusion

In this thesis, a method to count the vehicles from low-resolution images based on density maps is presented. Although there are more than one predictions for the larger vehicles, the algorithm is able to predict small and occluded vehicles. Larger vehicles can be detected by using the YOLO [10] model, and the results can be combined to predict locations of all the vehicles in an image. YOLO does not detect small objects which is why it cannot be used on its own. The processing time for each image is less than a second which makes it quite practical to use. A license plate localization algorithm is also presented which is based on the same network architecture used for counting vehicles.

Bibliography

[1] Paulo R.L. de Almeida et al. "PKLot – A robust dataset for parking lot classification". In: *Expert Systems with Applications* 42.11 (2015), pp. 4937 –4949. ISSN: 0957-4174. DOI: https://doi.org/10.1016/j.eswa.2015.02.009. URL: http://www.sciencedirect.com/science/article/pii/S0957417415001086.

[2] Giuseppe Amato et al. "Car parking occupancy detection using smart camera networks and deep learning". In: *Computers and Communication (ISCC), 2016 IEEE Symposium on*. IEEE. 2016, pp. 1212–1217.

[3] Giuseppe Amato et al. "Deep learning for decentralized parking lot occupancy detection". In: *Expert Systems with Applications* 72 (2017), pp. 327–334.

[4] *Complex Planet Painting*. Gravity George Artist. URL: https://www.gravitygeorge.co.nz/complex-planet/ (visited on 10/10/2018).

[5] Kun-Chan Lan and Wen-Yuah Shih. "An intelligent driver location system for smart parking". In: *Expert Systems with Applications* 41.5 (2014), pp. 2443 –2456. ISSN: 0957-4174. DOI: https://doi.org/10.1016/j.eswa.2013.09.044. URL: http://www.sciencedirect.com/science/article/pii/S0957417413007987.

[6] Rayson Laroca et al. "A Robust Real-Time Automatic License Plate Recognition based on the YOLO Detector". In: *CoRR* abs/1802.09567 (2018). arXiv: 1802.09567. URL: http://arxiv.org/abs/1802.09567.

[7] Syed Zain Masood et al. "License Plate Detection and Recognition Using Deeply Learned Convolutional Neural Networks". In: *CoRR* abs/1703.07330 (2017). arXiv: 1703.07330. URL: http://arxiv.org/abs/1703.07330.

[8] Daniel Oñoro-Rubio and Roberto J. López-Sastre. "Towards Perspective-Free Object Counting with Deep Learning". In: *Computer Vision – ECCV 2016*. Ed. by Bastian Leibe et al. Cham: Springer International Publishing, 2016, pp. 615–629.

[9] Esmat Rashedi and Hossein Nezamabadi-pour. "A hierarchical algorithm for vehicle license plate localization". In: *Multimedia Tools and Applications* 77.2 (2018), pp. 2771–2790. ISSN: 1573-7721. DOI: 10.1007/s11042-017-4429-z. URL: https://doi.org/10.1007/s11042-017-4429-z.

[10] Joseph Redmon et al. "You Only Look Once: Unified, Real-Time Object Detection". In: *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 779–788.

[11] Roberto López-Sastre Saturnino Maldonado Bascón Ricardo Guerrero-Gómez-Olmedo Beatriz Torre-Jiménez and Daniel Oñoro-Rubio. "Extremely Overlapping Vehicle Counting". In: *Iberian Conference on Pattern Recognition and Image Analysis (IbPRIA)*. 2015.

[12] S. Mohamed Mansoor Roomi, M. Anitha, and R. Bhargavi. "Accurate license plate localization". In: *2011 International Conference on Computer, Communication and Electrical Technology (ICCCET)*. 2011, pp. 92–97. DOI: 10.1109/ICCCET.2011.5762445.

[13] Jong-Ho Shin and Hong-Bae Jun. "A study on smart parking guidance algorithm". In: *Transportation Research Part C: Emerging Technologies* 44 (2014), pp. 299 –317. ISSN: 0968-090X. DOI: https://doi.org/10.1016/j.trc.2014.04.010. URL: http://www.sciencedirect.com/science/article/pii/S0968090X14001077.

[14] *Spring Backgrounds*. Wallpaper Cave. URL: https://wallpapercave.com/w/z0WMNbf (visited on 08/10/2018).

[15] *Wikipedia contributors*. The Free Encyclopedia. URL: https://commons.wikimedia.org/w/index.php?curid=45673581 (visited on 11/14/2018).